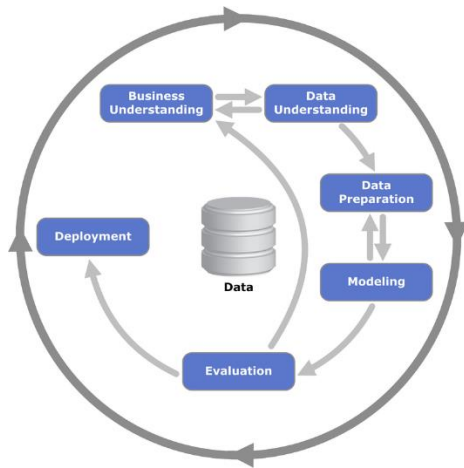


## Documentación Prueba Técnica- Daniel Alvarez Sanchez

La metodología a utilizar para el desarrollo del problema es CRISP\_DM, a continuación se procede a explicar cada uno de los pasos:



1. **Entendimiento del Negocio:** Hoy Bancolombia no tiene la capacidad de predecir con anticipación qué clientes en mora aceptarán una opción de pago. Esto limita la eficiencia en la priorización de la gestión de cobros, lo que afecta la efectividad en la reducción de la cartera vencida, para esto se requiere crear un modelo que logre predecir cuales clientes van a aceptar o no mi opción de pago.

### 2. Preparación de los datos:

#### Análisis de variables individual

En el primer paso se pasa al análisis de las variables de cada base de datos, y también comprender que tipos de datos nos están proporcionando y que objetivo cumplen con mi set de datos

- **prueba\_op\_master\_customer\_data\_enmascarado\_completa**

Esta base corresponde a características Analizo el tipo de variable de cada columna para convertirlos al que debería tener:

|                |        |
|----------------|--------|
| nit_enmascarar | int64  |
| cod_tipo_doc   | int64  |
| tipo_cli       | object |
| ctrl_terc      | object |
| genero_cli     | object |
| ano_nac_cli    | object |
| edad_cli       | object |
| estado_civil   | object |
| tipo_vivienda  | object |
| num_hijos      | object |
| personas_dep   | object |
| nivel_academ   | object |
| ocup           | object |
| act_econom     | object |
| sector         | object |
| subsector      | object |
| declarante     | object |

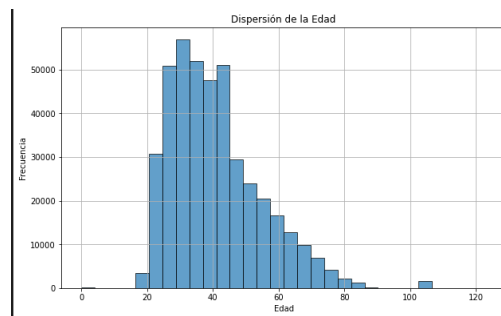
Según la metadata y el primer análisis de los datos vemos que hay columnas que no corresponden al esperado, para esto se realiza un cambio para luego hacer un análisis descriptivo para cada variable, ejemplo: edad\_cli, personas\_dependientes , total\_ing (objeto a numérico) , num\_hijos,tot\_patrimonio,tot\_activos,tot\_pasivos. Luego de esto realizamos un análisis descriptivo donde logramos ver comportamiento o valores atípicos de los datos.

Uno de los casos vistos es en el número de hijos donde se logra observar un valor atípico de 96 hijos, o edades mínimas de 0 , estos datos son importantes tenerlos en cuenta ya que si se utilizan en el modelo pueden generar ruido y afectar el resultado del modelo de predicción, para esto mas adelante se buscara métodos de imputación de datos, otra opción para comprender mis datos puede ser gráficamente , uno de los ejemplos es la edad donde se logran observar gente menor a 18 años y gente mayor a 100 años.

Campos con datos faltantes:

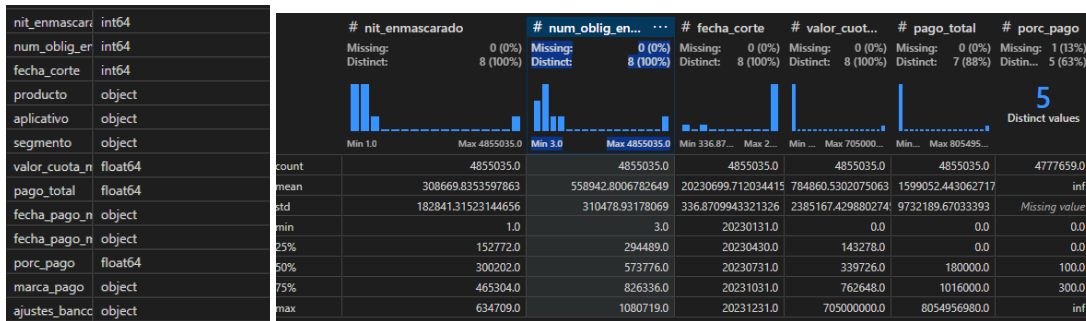
```
missing_values = df2.isnull().sum()
print(missing_values)

✓ 0.4s
nit_enmascarado      0
cod_tipo_doc         0
tipo_cli             0
ctrl_terc            0
genero_cli           0
ano_nac_cli          0
edad_cli             7942
estado_civil         0
tipo_vivienda        0
num_hijos            7942
personas_dependientes 7945
```



- **prueba\_op\_maestra\_cuotas\_pagos\_mes\_hist\_enmascarado\_completa**

Esta base representa los pagos a nivel de obligación que han tenido los clientes, donde el tipo de variables solo se ve afectado para los campos de fecha\_de\_apgo\_minimo y fecha\_de\_campo\_maximo. También logramos mirar la descripción de cada variable, aquí analizamos que nuestra fuente está para todo el año 2023, se tiene valores de cuota con valores muy extremos al igual que el pago total, para resolver si este corresponde a un dato atípico se tendría que explorar la obligación e indagar con expertos para ver su normalidad.



También se logra observar en el porc\_pago tiene valores inf, estos datos al revisarlos con más detalle se comprueban que el valor de la cuota es 0 pero tuvo un pago, eso se puede comprender como un dato erróneo o se puede indagar si hay alguna particularidad para entender mejor este campo.

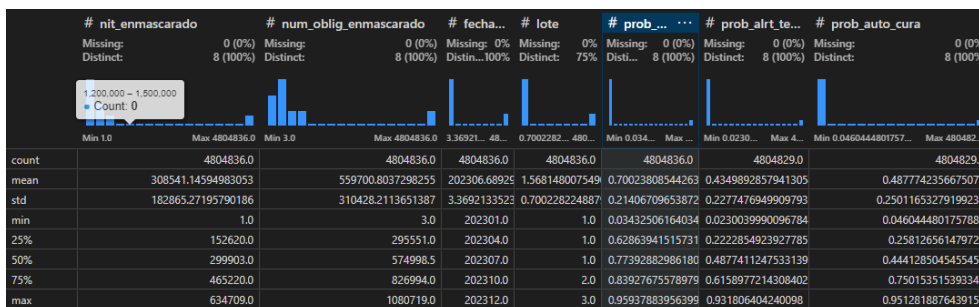
También se analiza la completitud de mis datos para saber si existen variables a eliminar por la calidad, en este caso vemos que existen campos con valores nulos, sin embargo, su porcentaje no es significativo en cuanto al total

```
missing values = df4.isnull().sum()
print(missing_values)
```

```
nit_enmascarado      0
num_oblig_enmascarado 0
fecha_corte          0
producto             0
aplicativo           0
segmento             0
valor_cuota_mes      0
pago_total           0
fecha_pago_minima    0
fecha_pago_maxima    0
porc_pago            77376
marca_pago           0
ajustes_banco        0
dtype: int64
```

### • prueba\_op\_maestra\_cuotas\_pagos\_mes\_hist\_enmascarado\_completa

Esta base representa la salida para cada obligación de distintos modelos de Alerta temprana, Auto cura, Propensión de pago, estas variables pueden ser significativas ya que pueden representar la intención o situación del cliente, sin embargo, esto puede tener varias dificultades puesto que puede existir una **dependencia de errores de los modelos anteriores, Sobreajuste** ya que podemos incluir variables correlacionadas o una **perdida de interpretabilidad**.



Con esto logramos observar que mis variables al ser una proporción se encuentran entre los valores esperado (0 -1) y no existen valores faltantes.

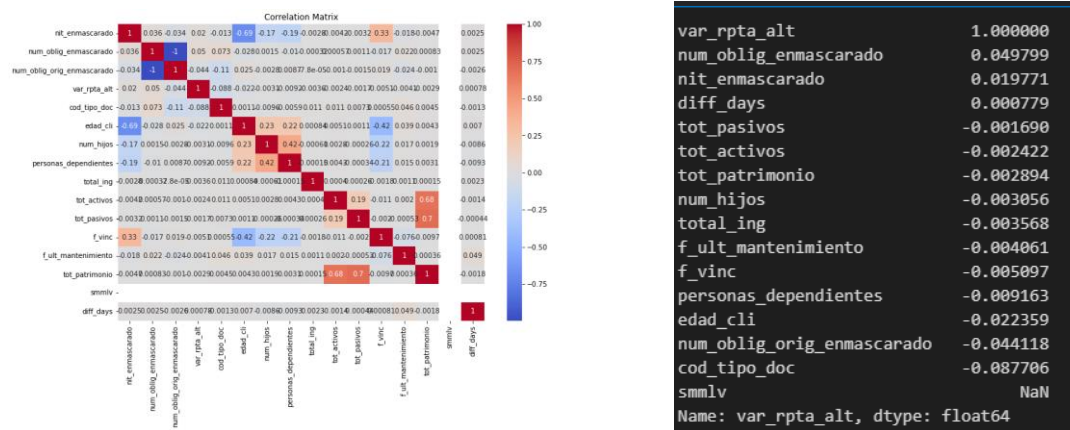
## Selección de Datos y creación de variables

El ejercicio parte de unas variables bases, las cuales intentamos buscar en nuestras bases de datos para incluirlas.

Para la selección de datos procuramos observar las variables que nos llamen mas la atención y tengan alguna relevancia con las variables respuesta, para esto se hace cruces para mirar si existe alguna correlación o si hay algún patrón a identificar.

- **prueba\_op\_master\_customer\_data\_enmascarado\_completa**

Para esta base se selecciona se le agrega a mis variable respuesta, las características de cada cliente en el momento de observación, ejemplo: para una respuesta dada en Julio del 2023, se analizaría la información que tiene el cliente en junio del 2023 y así lograr encontrar correlación de variables:

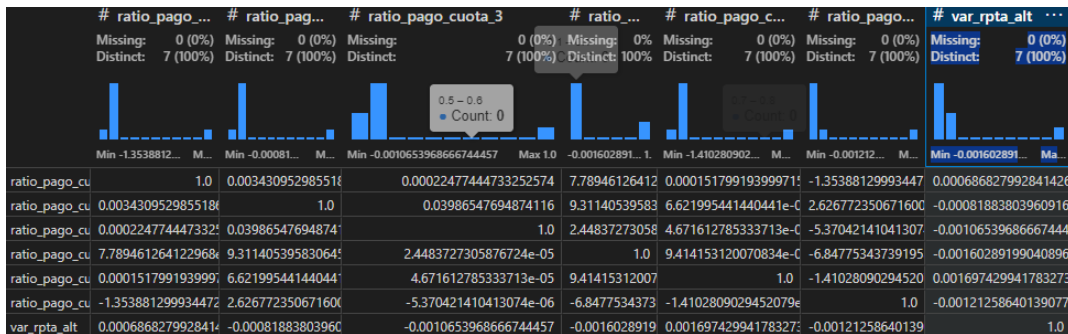


Para esta base se selecciona la edad del cliente que es la variable va en sentido opuesto de la respuesta, y puede agregar información a mi modelo y podemos incluir la variable segmento que está en el df de test, sin embargo, este no cuenta con una completitud, lo cual se opta por tomarlo de otra base.

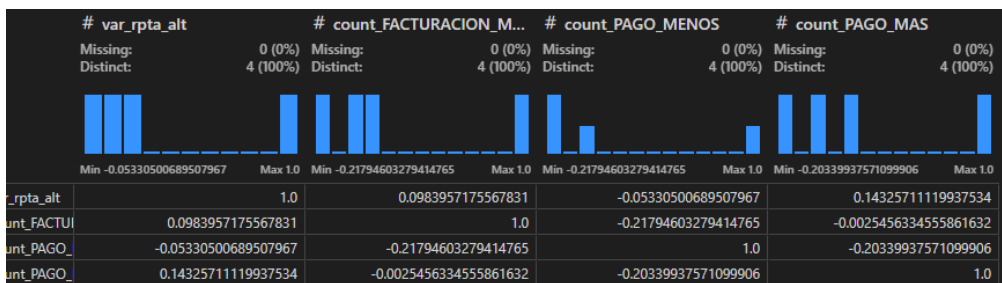
- **prueba\_op\_maestra\_cuotas\_pagos\_mes\_hist\_enmascarado\_completa**

Para la base de pagos el enfoque dado es analizar el comportamiento que ha tenido el cliente de manera histórica en sus pagos, si ha pagado o no sus cuotas, y poder observar algún patrón que logre indicar si va a aceptar la opción de pagos, para esto se tomó los últimos 6 meses desde la fecha de observación y se creo variables por cada mes de observación, se tuvo en cuenta el valor de la cuota y valor pagado, para crear nuevamente el porcentaje de pago.

Después de observar esto se analiza la relación con las variables salida:



También es importante se crea una variable con el contador de marca\_de\_pago esto puede dar un indicio de cómo se ha comportado el cliente en sus pagos según obligación.



Al finalizar no logramos ver que ninguna variable en pagos significativa, pero a parte de esto tomamos los campos que están en nuestro df\_test como pago\_cuota, valor total\_mes, segmento y el conteo de marca\_pago que nos puede lograr indicar una correlación con nuestra variable de salida.

Además, se crea el campo marca\_pago2, donde se tiene como regla si el cliente pago su cuota, si lo pago parcial, si no pago, o ya no existe, esto con el fin de tratar de recrear el campo marca\_pago del test.

```
# Calcular nuevamente porc_pago si es nulo
merged_df['porc_pago'] = merged_df.apply(
    lambda row: (row['pago_total'] / row['valor_cuota_mes']) * 100 if pd.isnull(row['porc_pago']) == 0 else row['porc_pago'],
    axis=1
)

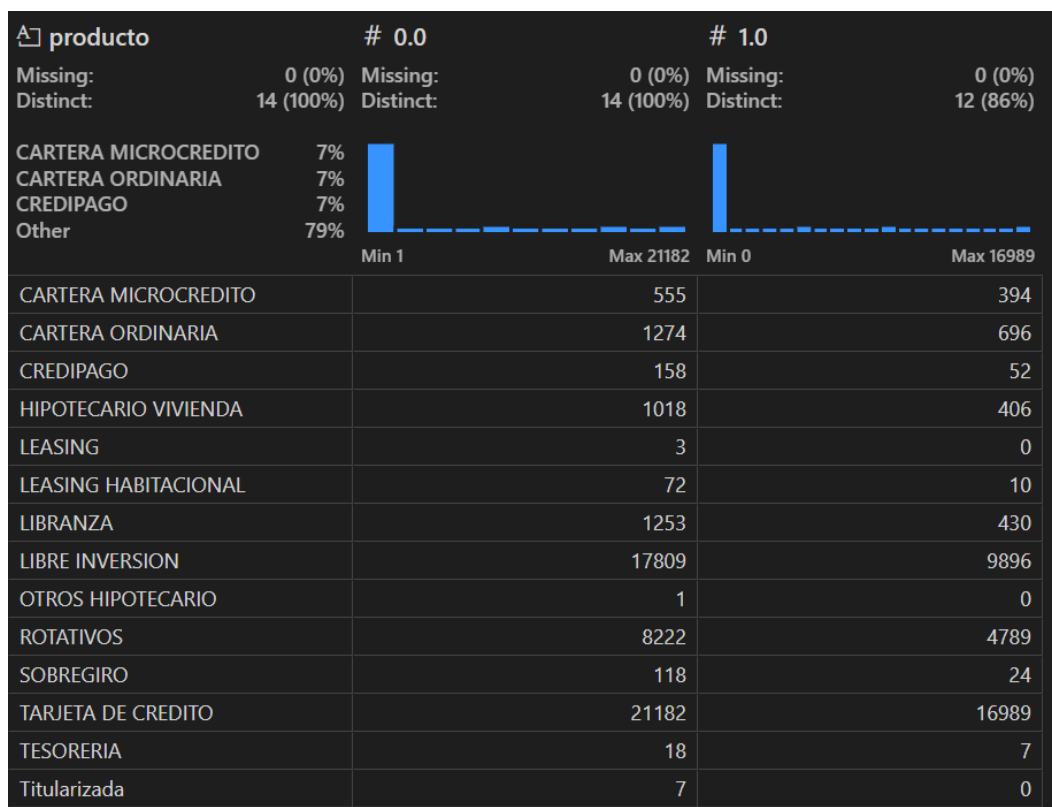
merged_df['marca_pago2'] = merged_df.apply(
    lambda row: 'paga mas' if row['pago_total'] > row['valor_cuota_mes'] else (
        'paga menos' if row['pago_total'] < row['valor_cuota_mes'] else (
            'pago completo' if row['pago_total'] == row['valor_cuota_mes'] else ''
        )
    ) if row['pago_total'] == 0 and row['valor_cuota_mes'] == 0 else 'cancelado',
    axis=1
)
```

Además, se vuelve a calcular el campo de porc\_pago, hay valores que son inf, estos valores se cambia a 1 ya que representa un abono ya que no existía deuda en la cuota.

Sin embargo, es bueno tener en cuenta si esto es correcto ya que no guarda mucha coherencia.

También de nuestra base principal de test, vemos que de esta base de pagos podemos extraer la información del cliente de producto, aplicativo, segmento.

Otro análisis que podemos realizar es mis variables categóricas que relación tienen con mi variable respuesta, para entender patrones, este análisis se hizo para cada variable seleccionada categórica:



Un ejemplo de esta tabla es que podemos ver que los libre inversión en su mayoría no aceptan la opción de pagos.

- **prueba\_op\_probabilidad\_oblig\_base\_hist\_enmascarado\_completa**

De esta base no logramos ver variables que tengan nuestro df test , sin embargo en el estudio es vemos importancia en ver si este puede representar alguna correlación con nuestra variable objetivo, para esto tomamos los últimos 6 valores para cada pronostico y observamos su correlación.

```
# Calcular la correlación entre var_rpta_alt y prob_propension_5
correlation_matrix = merged_check.corr()
var_rpta_correlation = correlation_matrix['var_rpta_alt']
print(var_rpta_correlation)
```

✓ 0.7s 🔗 Open 'var\_rpta\_correlation' in Data Wrangler

|                            |           |
|----------------------------|-----------|
| var_rpta_alt               | 1.000000  |
| prob_propension_5          | 0.189754  |
| prob_auto_cura_5           | 0.188926  |
| prob_auto_cura_4           | 0.187775  |
| prob_auto_cura_3           | 0.187186  |
| prob_propension_4          | 0.182742  |
| prob_auto_cura_2           | 0.177413  |
| prob_propension_3          | 0.176234  |
| prob_auto_cura_1           | 0.170468  |
| prob_propension_2          | 0.156741  |
| prob_auto_cura_6           | 0.154451  |
| prob_propension_6          | 0.152409  |
| prob_propension_1          | 0.142784  |
| num_oblig_enmascarado      | 0.049799  |
| nit_enmascarado            | 0.019771  |
| num_oblig_orig_enmascarado | -0.044118 |
| prob_alrt_temprana_6       | -0.131197 |
| prob_alrt_temprana_1       | -0.131829 |
| prob_alrt_temprana_2       | -0.151947 |
| prob_alrt_temprana_5       | -0.163647 |
| prob_alrt_temprana_4       | -0.167509 |
| prob_alrt_temprana_3       | -0.168739 |

Name: var\_rpta\_alt, dtype: float64

Con esto podemos ver que pueden generar un valor en nuestro modelo, pero para no sobredimensionar nuestro modelo solo se opta por tomar los valores de los modelos mas cercanos a la fecha de observación.

## Limpieza de datos

La limpieza de datos corregimos la calidad de los datos nulos, duplicados o inconsistentes

Que podamos tener en nuestras variables seleccionadas, en todo mi df armado logramos observar valores faltantes o atípicos.

- **prueba\_op\_master\_customer\_data\_enmascarado\_completa**

Aunque no se selecciona ninguna variable de este df, podemos limpiar la edad, mediante la mediana de los datos, y se convertiría no solo los datos faltantes , si no lo datos atípicos como menores de 18 o mayores de 100.

Y en el segmento podemos realizar una imputación con la moda de los datos, sin embargo otro método puede ser buscar características paracidas de los clientes, para imputar los datos.

- **prueba\_op\_maestra\_cuotas\_pagos\_mes\_hist\_enmascarado\_completa**

Los campos creados con la sumatoria de la marca\_de\_pagos se limpia mediante la asignación del valor 0, la premisa tomada es que los clientes que no tienen esta información no tienen registro de pagos, o pueden tener acuerdos donde no se haya presentado datos.

Además, se vuelve a calcular el campo de porc\_pago, hay valores que son inf, estos valores se cambian a 1 ya que representa un abono ya que no existía deuda en la cuota.

Los campos de segmento, aplicativo, producto se imputan por el método de KNNImputer que logra observar características (vecinos) parecidos en otras variables y asignarle un valor cercano.

```
from sklearn.impute import KNNImputer

# Define the columns to exclude from imputation
exclude_columns = ['nit_enmascarado', 'num_oblig_orig_enmascarado', 'num_oblig_enmascarado']

# Select the columns to impute
columns_to_impute = final_merged_df.columns.difference(exclude_columns)

# Initialize the KNNImputer
imputer = KNNImputer()
```

```
# Reemplazar valores infinitos en la columna 'porc_pago' con 'missing'
final_merged_df['porc_pago'] = final_merged_df['porc_pago'].replace([float('inf'), float('-inf')], float('nan'))

# Impute the missing values
final_merged_df[columns_to_impute] = imputer.fit_transform(final_merged_df[columns_to_impute])
```

Sin embargo para temas de simplicidad en el ejercicio , utilizamos la función simpleImputer()

```
from sklearn.impute import SimpleImputer

# Imputar variables categóricas con la moda
categorical_columns = final_merged_df.select_dtypes(include=['object']).columns
categorical_imputer = SimpleImputer(strategy='most_frequent')
final_merged_df[categorical_columns] = categorical_imputer.fit_transform(final_merged_df[categorical_columns])

# Imputar variables numéricas con la media
numerical_columns = final_merged_df.select_dtypes(include=['float64', 'int64']).columns
numerical_imputer = SimpleImputer(strategy='mean')
final_merged_df[numerical_columns] = numerical_imputer.fit_transform(final_merged_df[numerical_columns])
```

## Preparación de los datos

Después de definir mis variables a evaluar a prepara mis datos para que nuestro modelo lo comprenda, después de la limpieza, mis variables categóricas se convierten en variables dummies.



```
# Crear variables dummies para las columnas categóricas en final_merged_df
final_merged_df = pd.get_dummies(final_merged_df, columns=['producto', 'aplicativo', 'segmento', 'marca'])
```

✓ 0.6s Python

Otro paso puede ser la normalización de los datos, esto para que mis datos con grandes escalas no afecten el resultado de mi modelo.

```
from sklearn.preprocessing import StandardScaler

# Definir las características (X) y la variable objetivo (y)
X = final_merged_df.drop(columns=['nit_enmascarado', 'num_oblig_orig_enmascarado', 'num_oblig_enmascarado'])
y = final_merged_df['porc_pago']

# Dividir los datos en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Inicializar el escalador
scaler = StandardScaler()
```

## Balanceo de datos

Mis datos no presentan un desbalanceo significativo que pueda llevar al sobre entrenamiento del modelo, por lo que no es necesario aplicar técnicas de balanceo como SMOTE.

```
conteo_var_rpta = final1['var_rpta_alt'].value_counts()
print(conteo_var_rpta)
```

✓ 0.0s Open 'conteo\_var\_rpta' in Data Wrangler

```
0    295483
1    272768
Name: var_rpta_alt, dtype: int64
```

## 3. Modelamiento y Evaluación

Para el modelamiento utilizamos la metodología de champion challenger, que básicamente es colocar a competir distintos modelos para lograr escoger el mejor.

```
# Definir los modelos a evaluar
models = {
    'RandomForest': RandomForestClassifier(),
    'LogisticRegression': LogisticRegression(max_iter=1000),
    'GradientBoosting': GradientBoostingClassifier(),
    'DecisionTree': DecisionTreeClassifier(),
    'XGBoost': xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')
}
```

```
termino modelo LogisticRegression
termino modelo GradientBoosting
termino modelo DecisionTree
termino modelo XGBoost
Model: RandomForest
Accuracy: 0.7921
F1 Score: 0.7730
ROC AUC: 0.8677
```

```
Model: LogisticRegression
Accuracy: 0.7095
F1 Score: 0.6974
ROC AUC: 0.7727
```

```
Model: GradientBoosting
Accuracy: 0.7591
F1 Score: 0.7376
ROC AUC: 0.8298
```

```
Model: DecisionTree
Accuracy: 0.7075
F1 Score: 0.6954
ROC AUC: 0.7071
```

```
Model: XGBoost
...
F1 Score: 0.7615
ROC AUC: 0.8564
```

Después de tomar nuestro mejor modelo, nos apoyamos en la búsqueda de los hiperparámetros de este, ya que con esto podemos mejorar la evaluación de nuestro modelo y encontrar uno con mejor predicción.

```
from sklearn.model_selection import GridSearchCV
# Definir el modelo
xgb_model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss')
# Definir los hiperparámetros a buscar
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 8],
    'learning_rate': [0.01, 0.2],
    'subsample': [0.8, 1.0]
}
# Configurar GridSearchCV
grid_search = GridSearchCV(estimator=xgb_model, param_grid=param_grid, cv=2, scoring='roc_auc', verbose=1)
# Ejecutar GridSearchCV
grid_search.fit(X_train, y_train)
```

```
print(f'Accuracy: {accuracy}')
print(f'F1 Score: {f1}')
print(f'ROC AUC Score: {roc_auc}')
```

✓ 0.0s

Accuracy: 0.6718198696007954

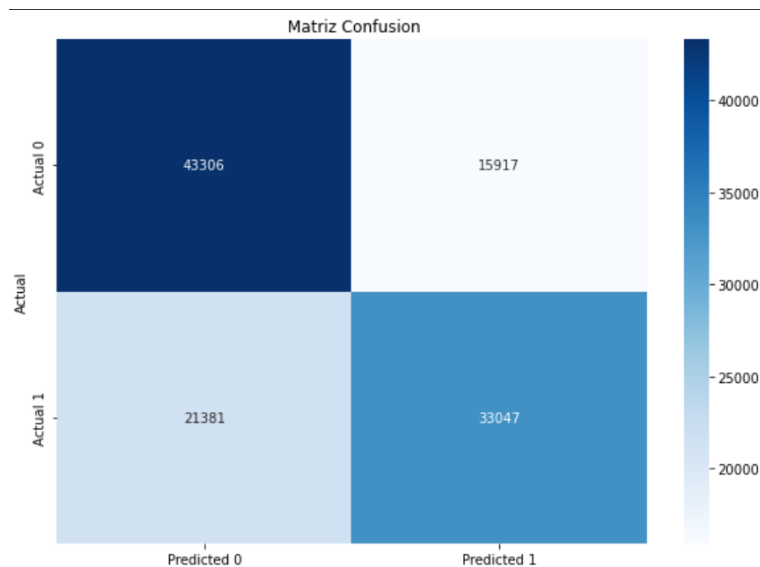
F1 Score: 0.639256422160322

ROC AUC Score: 0.7355542082925552

Ya después de nuestro modelamiento logramos realizar nuestro entrenamiento final y nuestro test sacando.

Para entender la evaluación de nuestro modelo nos disponemos a entender nuestras métricas.

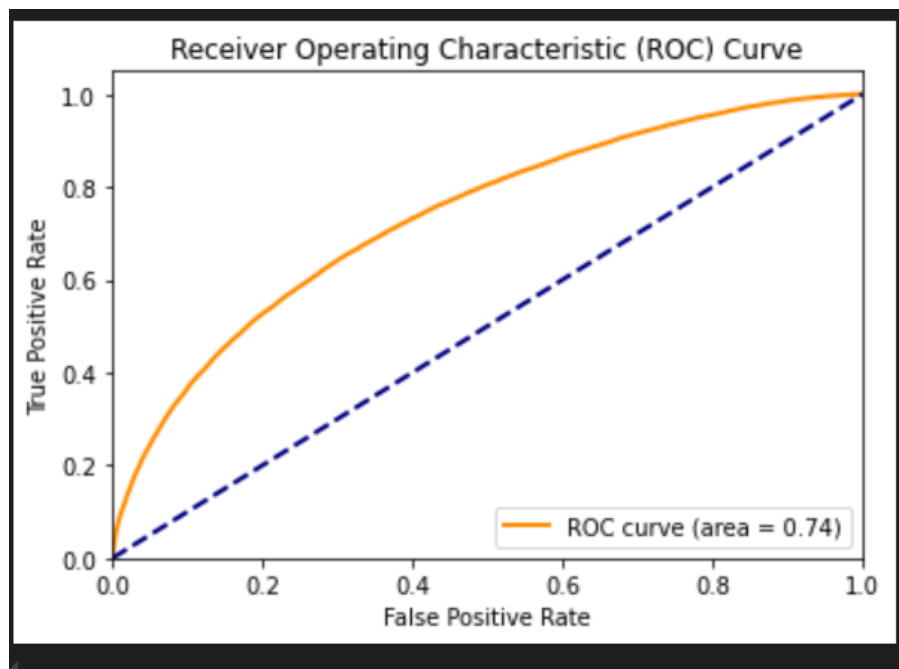
Todas estas parten desde mi matriz de confusión donde puedo ver la clasificación de cada uno de mis categorías de datos: ++, +-, --, -+



**precisión:** es la proporción de predicciones correctas (tanto verdaderos positivos como verdaderos negativos)

**F1- Score:** El F1 score es una métrica que combina la precisión y el recall en una sola medida.

**ROC:** es un gráfico que muestra la tasa de verdaderos positivos frente a la tasa de falsos positivos.



En conclusión, no vemos un gran performance de nuestro modelo esta tendría que ser mas cercana a 1.

Para la predicción de mis nuevos datos tengo que tener en cuenta que pueden existir campos en mis variables categóricas que no existan, ejemplo :productos.

Tengo que asegurar que mi datos a predecir mantienen la misma estructura que mis datos de entrenamiento, para esto al hacer la transformación de mis datos a predecir encuentro las columnas que no están comparando con mi datos de entrenamiento y las agrego, inicializadas en 0 ya que no existen, y de igual manera si existe un producto nuevo lo tengo que eliminar ya que mi modelo no lo entiende.

```
# Obtener las columnas que están en final1 pero no en final_merged_df, excluyendo 'var_rpta_al'
columns_to_add = set(final1.columns) - set(final_merged_df.columns) - {'var_rpta_al'}

# Crear las columnas faltantes en final_merged_df y llenarlas con ceros
for column in columns_to_add:
    final_merged_df[column] = 0
```

✓ 0.0s Python

```
# Obtener las columnas que están en final_merged_df pero no en final1
columns_to_drop = set(final_merged_df.columns) - set(final1.columns)

# Eliminar las columnas de final_merged_df
final_merged_df = final_merged_df.drop(columns=columns_to_drop)
```

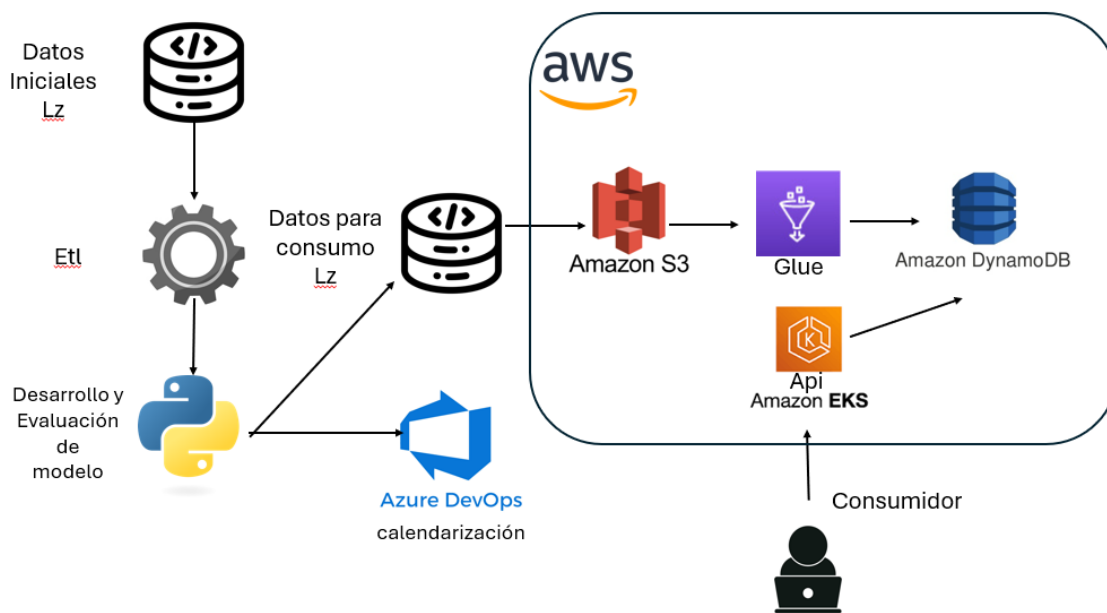
✓ 0.0s Python

#### Análisis de otras variables:

Otro tipo de variables que indiquen históricamente la condición de nuestro cliente, como por ejemplo si este ha tenido

- **reestructuras en sus créditos.**
- **Información de ley de insolvencia**
- **Información histórica de sus créditos: mora, pagos, anticipos.**

## 4. Despliegue



La productización de mi modelo se implementaría dentro del ecosistema bancario, comenzando con la ingesta de datos desde la lz un área dedicada al almacenamiento inicial de la información en bruto. Una vez obtenidos, se realizaría un proceso de ETL, utilizando SQL para la extracción de datos y Python para la transformación y limpieza de la información, asegurando que los datos estén listos para el análisis y el modelado.

Posteriormente, se procederá al desarrollo de los modelos analíticos, realizando una serie de pruebas y evaluaciones para optimizar su rendimiento. Tras este proceso, el modelo se calendarizará, estableciendo tiempos definidos para su ejecución de manera automatizada y continua. Los resultados obtenidos se visualizarán en las zonas de destino dentro de la lz, donde los equipos correspondientes podrán acceder a ellos.

El siguiente paso consistirá en integrar todo el flujo de datos y resultados dentro del ecosistema de AWS. Usaré Amazon S3 para el almacenamiento escalable y seguro de los datos en la nube, mientras que AWS Glue se utilizará para la integración y el procesamiento de datos a gran escala. Los resultados y datos transformados se almacenarán en Amazon DynamoDB, aprovechando su alta disponibilidad y escalabilidad para consultas rápidas y eficientes.

Finalmente, para facilitar el consumo de los modelos y sus resultados por parte de otros sistemas, exponer los modelos mediante una API será el siguiente paso. Utilizaré Amazon EKS para gestionar la API. Esto permitirá que otras aplicaciones o usuarios puedan acceder a los modelos y consumir sus predicciones en tiempo real.